

Benchmark y Rendimiento de Aplicaciones Paralelas

Caso 4 — Computación Paralela y Distribuida, Sección 20. Universidad del Valle de Guatemala. Equipo: Luis Palacios, Diego, Roberto, Nico y Esteban.

1. Introducción y objetivo

El número que aparece en las noticias cuando una supercomputadora rompe un récord no es el rendimiento que esa máquina entrega en aplicaciones reales. Partimos de esa observación para cumplir el objetivo del caso: realizar pruebas comparativas en un sistema paralelo con varios benchmarks, identificar los cuellos de botella y proponer optimizaciones. Investigamos y comparamos los benchmarks estándar del área (LINPACK, HPCG y, de forma teórica, SPEC), medimos el ancho de banda con STREAM y construimos dos aplicaciones propias para observar los extremos del comportamiento. Todo el análisis gira en torno a una brecha concreta y medible entre el rendimiento pico y el rendimiento real.

2. Marco teórico

Nuestro trabajo se apoya en tres pilares de la literatura. Primero, el «muro de memoria»: Wulf y McKee (1995) advirtieron que la velocidad del procesador y la de la memoria mejoran a ritmos distintos, de modo que la brecha crece de forma exponencial y el procesador termina esperando a la memoria. Segundo, el modelo Roofline de Williams, Waterman y Patterson (2009), que grafica el rendimiento frente a la intensidad aritmética (operaciones por byte) y hace visibles dos techos: el ancho de banda y la capacidad de cómputo. Tercero, la justificación de HPCG de Dongarra, Heroux y Luszczek (2016), quienes crearon ese benchmark porque HPL dejó de predecir el rendimiento de las aplicaciones reales. El ancho de banda sostenible lo enmarca McCalpin (1995) con STREAM.

3. Metodología

Ejecutamos todas las corridas en una sola máquina de referencia —una MacBook con chip Apple M4 Pro (12 núcleos, 24 GB de memoria unificada)— dentro de un contenedor Docker, para que los números fueran comparables entre sí y reproducibles en cualquier sistema operativo. Para cada benchmark barrimos 1, 2, 4 y 8 hilos o procesos, y registramos GFLOP/s, tiempo, speedup y ancho de banda. Comparamos seis cargas:

- HPL / LINPACK — álgebra lineal densa; expone el límite de cómputo (Dongarra, Luszczek y Petitet, 2003).
- HPCG — gradiente conjugado con matriz dispersa; expone el límite de memoria.
- STREAM — ancho de banda de memoria sostenible.
- NPB (NAS Parallel Benchmarks) — kernels CG y EP, clases A/B/C, en OpenMP y MPI (Bailey et al., 1994).
- Aplicaciones propias — una simulación N-body gravitacional (compute-bound) y un stencil de calor 3D por Jacobi (memory-bound).

Aclaración importante: no ejecutamos SPEC porque es de licencia paga; lo cubrimos a nivel teórico y lo sustituimos en la práctica por las NAS Parallel Benchmarks, que son gratuitas y miden el mismo tipo de cargas variadas.

4. Resultados

La brecha del pico. Confirmamos con datos oficiales que el sistema #1 del mundo en noviembre de 2025, El Capitan, promete 2 821 PFlop/s teóricos (Rpeak), alcanza 1 809 PFlop/s medidos en HPL (Rmax) y cae a solo 17.41 PFlop/s en HPCG: menos del 1 % de su pico, es decir, 104 veces más lento consigo mismo (TOP500, 2025). Nos dimos cuenta de que el fenómeno no depende de la escala: en nuestra Mac reproducimos la misma forma, con HPL en 122 GFLOP/s y HPCG en 6.5 GFLOP/s —una brecha de 19×.

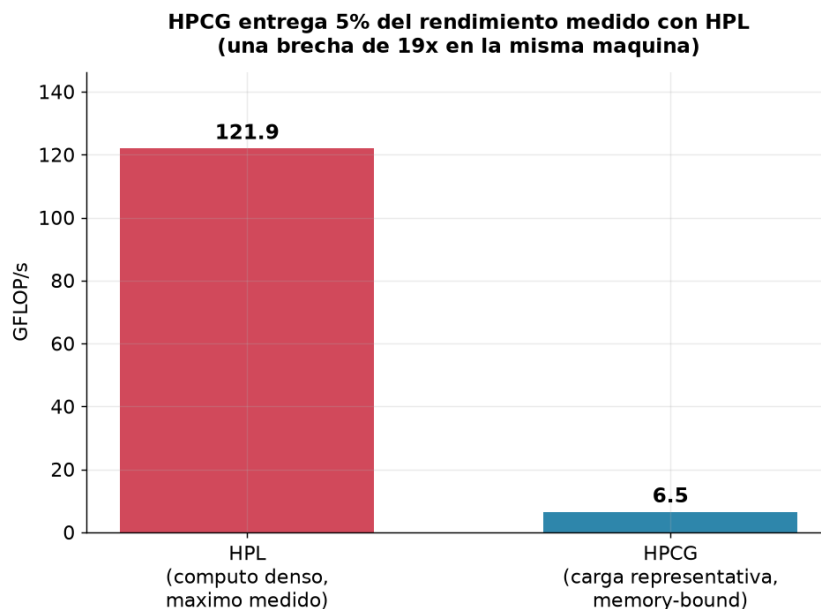


Figura 1. La misma máquina rinde ~5 % en una carga realista (HPCG) frente a HPL.

El Roofline. Al ubicar cada carga en el modelo Roofline vimos con claridad contra qué techo choca. Usamos como techo de memoria el pico teórico publicado del M4 Pro (273 GB/s; Apple, 2024) —una fuente independiente de nuestras mediciones— y como techo de cómputo el máximo medido (128 GFLOP/s). HPL y nuestra N-body quedan pegadas al techo de cómputo, mientras que HPCG y nuestro stencil caen en la zona de memoria; el stencil sostiene alrededor del 58 % del ancho de banda pico.

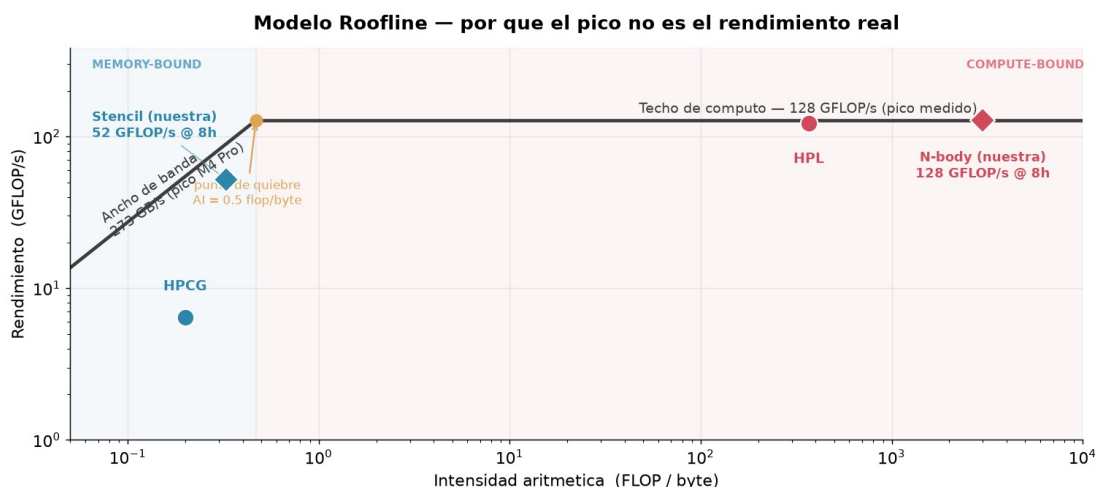


Figura 2. Modelo Roofline: cada benchmark choca contra un techo distinto (cómputo o memoria).

El costo de más núcleos. Medimos que la carga compute-bound (N-body) escala $5.6\times$ de 1 a 8 hilos —casi ideal—, mientras que la memory-bound (stencil) se estanca en $3.6\times$ y deja de mejorar tras cuatro hilos: al saturar el ancho de banda, los núcleos adicionales quedan esperando datos. Además, al comparar el mismo kernel de NPB en OpenMP y MPI observamos un rendimiento casi idéntico, porque en un solo nodo MPI usa transporte de memoria compartida y el costo de comunicación es mínimo.

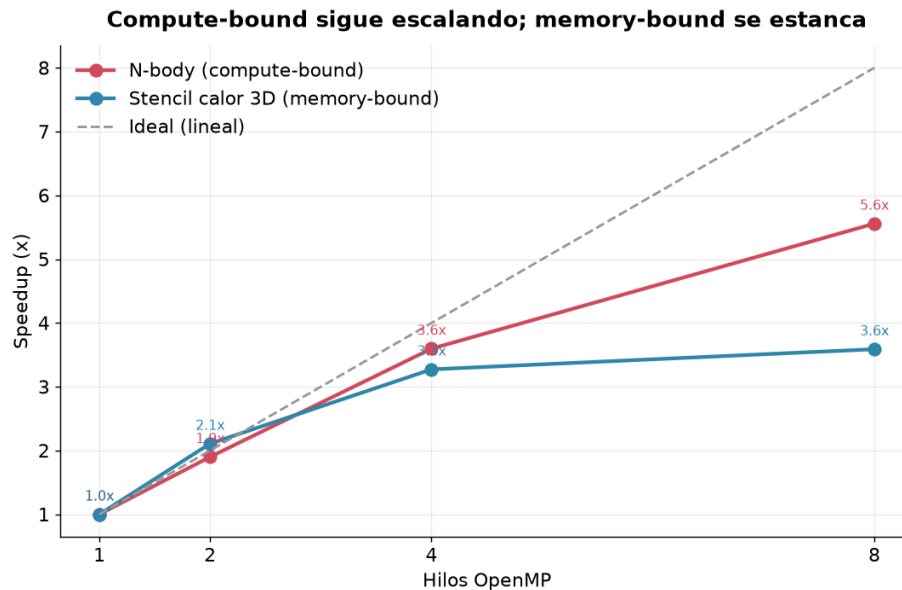


Figura 3. Escalamiento: la carga compute-bound sigue escalando; la memory-bound se estanca.

5. Cuellos de botella identificados

- Ancho de banda de memoria — HPCG y el stencil (intensidad aritmética ≈ 0.33 flop/byte) saturan la memoria mucho antes que la unidad de cálculo.
- Escalabilidad limitada por memoria — el stencil no mejora tras cuatro hilos; los núcleos extra no aportan.
- Overhead de comunicación — despreciable en un nodo, pero dominante al escalar a varios nodos con red.
- Pico frente a sostenido — ni HPL alcanza más del 64 % del pico teórico (Rmax/Rpeak de El Capitan).
- Thermal throttling — en una laptop, las corridas largas bajan de frecuencia por calor; lo documentamos como nota de método.

6. Propuestas de optimización

- Mejorar la localidad de datos con cache blocking / tiling y layout consciente de caché, para reducir el tráfico a memoria.
- No sobre-provisionar hilos en cargas memory-bound: usar el punto donde satura el ancho de banda y volcar el resto del esfuerzo a mejorar la localidad.
- Solapar cómputo y comunicación y usar mapeo consciente de topología cuando el límite sea la red.

- Elevar la intensidad aritmética apoyándose en librerías de alto reuso (BLAS nivel 3), para correr el techo del Roofline a nuestro favor.

7. Conclusiones

Comparamos los benchmarks, hallamos el cuello de botella y propusimos mejoras, cerrando el círculo del objetivo. Nuestra conclusión es directa: comprar más núcleos no elimina el cuello de botella. Primero hay que saber contra qué techo está chocando el código —y en las cargas realistas ese techo casi siempre es la memoria, no el cómputo. El rendimiento real no vive en el número pico, sino en el modelo Roofline.

Referencias

- Apple. (2024). Apple introduces M4 Pro and M4 Max. Apple Newsroom. <https://www.apple.com/newsroom/2024/10/apple-introduces-m4-pro-and-m4-max/>
- Bailey, D., Barszcz, E., Barton, J., Browning, D., Carter, R., Dagum, L., ... Weeratunga, S. (1994). The NAS Parallel Benchmarks (Technical Report RNR-94-007). NASA Ames Research Center.
- Dongarra, J., Heroux, M. A., & Luszczek, P. (2016). High-Performance Conjugate-Gradient benchmark: A new metric for ranking high-performance computing systems. *The International Journal of High Performance Computing Applications*, 30(1), 3–10. <https://doi.org/10.1177/1094342015593158>
- Dongarra, J., Luszczek, P., & Petitet, A. (2003). The LINPACK benchmark: Past, present and future. *Concurrency and Computation: Practice and Experience*, 15(9), 803–820. <https://doi.org/10.1002/cpe.728>
- McCalpin, J. D. (1995). Memory bandwidth and machine balance in current high performance computers. *IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter*, 19–25.
- TOP500. (2025). TOP500 list — November 2025. <https://top500.org/lists/top500/2025/11/>
- Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65–76. <https://doi.org/10.1145/1498765.1498785>
- Wulf, W. A., & McKee, S. A. (1995). Hitting the memory wall: Implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23(1), 20–24. <https://doi.org/10.1145/216585.216588>